

FCSC 2025 — Grand Classic Hotel

(MIFARE Classic)

Rapport de résolution

Grand Classic Hotel

The screenshot shows a web browser window displaying the Hackropole challenge page for 'Grand classic hotel'. The page is titled 'Grand classic hotel' and includes a 'hardware' tag and a 'résolu le 2025-11-12, 20:47:07' status. The description states that the user is a guest at the Grand Classic Hotel and has lost their Mifare Classic badge. The challenge involves identifying the ISO 14443-A exchanges, extracting the sector key, and retrieving the application data containing the flag. The 'Fichiers' section shows a file named 'grand-classic.trace'. The 'Auteur' section shows the user 'jle'. The 'Flag' section contains the flag 'FCSC{dca41baf48c57bf2c9309c485da267d23de04f9}'. The 'Soumettez votre solution' section shows a text input field with the URL 'https://gist.github.com/anssi-fr/4c13de1b2831555c6563462e5de6942d' and an 'Envoyer' button.

Résumé exécutif

Ce rapport documente la résolution du challenge FCSC 2025 « Grand Classic Hotel » à partir d'une trace Proxmark3 MIFARE Classic. L'objectif est d'identifier les échanges ISO 14443-A, d'extraire (ou retrouver) la clé de secteur MIFARE Classic, puis de récupérer les données applicatives contenant le flag.

Flag reconstruit : FCSC{dca41baf48c57bf2c9309c485da267d23de04f9}

1. Données d'entrée et outillage

- Trace Proxmark3 au format .trace : grand-classic.trace.

- Client Proxmark3/pm3 (mode OFFLINE) pour charger la trace, lister les trames et éventuellement déchiffrer.
- Optionnel : outils de récupération de clés MIFARE Classic (mfkey32, hardnested) si la clé n'est pas triviale.

2. Rappels MIFARE Classic utiles

Une carte MIFARE Classic 1K est organisée en 16 secteurs. Chaque secteur contient 4 blocs de 16 octets : 3 blocs de données et 1 bloc « sector trailer ». Le sector trailer contient Key A, les bits d'accès et Key B, et ne doit pas être confondu avec des données applicatives.

Le secteur 0 (blocs 0–3) contient notamment le bloc 0 « manufacturer » (UID + BCC + données constructeur), qui est particulier. Le premier secteur typiquement utilisé pour des données applicatives est donc le secteur 1, dont les blocs de données sont 4, 5 et 6 (le bloc 7 étant le trailer).

Dans la trace, l'information utile est précisément stockée dans les trois blocs de données du secteur 1, ce qui explique le focus sur les blocs 4–6.

3. Lecture de la trace dans pm3 (mode OFFLINE)

Les commandes ci-dessous se tapent dans l'interface interactive pm3. Important : ne pas copier/coller le prompt « [offline] pm3 --> » ; il faut saisir uniquement les commandes.

3.1 Charger la trace et vérifier ISO 14443-A

```
proxmark3
trace load -f grand-classic.trace
trace list -1 -t 14a
```

Cette étape permet de confirmer l'activation ISO14443-A (WUPA/REQA), l'anticollision et la sélection de la carte (UID, ATQA, SAK).

3.2 Lister les échanges MIFARE Classic

```
trace list -1 -t mf
```

La vue MIFARE Classic met en évidence les commandes AUTH (0x60/0x61) et READBLOCK (0x30). Si la trace est chiffrée côté MIFARE Classic, il faut déchiffrer avant de relister.

3.3 Déchiffrer la trace (si nécessaire)

```
trace decrypt -k FFFFFFFFFFFF
trace list -1 -t mf
```

Dans beaucoup de scénarios CTF, le secteur utilisé est protégé par une clé par défaut (ex. FFFFFFFFFFFF). Si la clé n'est pas connue, il faut la dériver à partir des nonces AUTH (cf. §5).

3.4 Captures d'écran (exemple de listing)

Les captures suivantes illustrent typiquement la sortie de « trace list -1 -t mf » et la présence des READBLOCK ciblés.

```
kali@kali: /mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel

kali@kali:~$ proxmark3
[-] Session log /home/kali/.proxmark3/logs/log_20260105190025.txt
[-] loaded /home/kali/.proxmark3/preferences.json
[-] Offline mode. Check "proxmark3 -h" if it's not what you want.

      8888888b.  888b  d888  .d88888b.
      888  Y88b 88888b  d8888  d88P  Y88b
      888  888 888888b d88888  .d88P
      888  d88P 888Y88888P888  8888"
      8888888P" 888 Y88P 888  "Y8b.
      888  888  Y8P  888 888  888
      888  888  "  888 Y88b  d88P
      888  888  888  "Y888P"  [ + ]

Release v4.18994 - Backdoor

[ Invest in open innovation! ]
Patreon - https://www.patreon.com/iceman1001/
Paypal - https://www.paypal.me/iceman1001/

[ Proxmark3 RFID instrument ]

[offline] pm3 -> [offline] pm3 -> trace load -f grand-classic.trace

help      Use '<command> help' for details of a command
prefs     { Edit client/device preferences... }
----- Technology -----
analyse   { Analyse utils... }
data      { Plot window / data buffer manipulation... }
emu       { EMV ISO-14443 / ISO-7816... }
hf        { High frequency commands... }
hw        { Hardware commands... }
lf        { Low frequency commands... }
nfc       { NFC commands... }
piv       { PIV commands... }
reveng    { CRC calculations from RevEng software... }
smart     { Smart card ISO-7816 commands... }
script    { Scripting commands... }
trace     { Trace manipulation... }
wiegand   { Wiegand format manipulation... }
----- General -----
clear     Clear screen
hints     Turn hints on / off
msleep    Add a pause in milliseconds
rem       Add a text line in log file
quit      Exit program
exit
```

```
kali@kali: /mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel

[offline] pm3 -> [offline] pm3 -> trace list -t mf

help      Use '<command> help' for details of a command
prefs     { Edit client/device preferences... }
----- Technology -----
analyse   { Analyse utils... }
data      { Plot window / data buffer manipulation... }
emu       { EMV ISO-14443 / ISO-7816... }
hf        { High frequency commands... }
hw        { Hardware commands... }
lf        { Low frequency commands... }
nfc       { NFC commands... }
piv       { PIV commands... }
reveng    { CRC calculations from RevEng software... }
smart     { Smart card ISO-7816 commands... }
script    { Scripting commands... }
trace     { Trace manipulation... }
wiegand   { Wiegand format manipulation... }
----- General -----
clear     Clear screen
hints     Turn hints on / off
msleep    Add a pause in milliseconds
rem       Add a text line in log file
quit      Exit program
exit

[offline] pm3 -> [offline] pm3 -> trace decrypt -k FFFFFFFFFF

help      Use '<command> help' for details of a command
prefs     { Edit client/device preferences... }
----- Technology -----
analyse   { Analyse utils... }
data      { Plot window / data buffer manipulation... }
emu       { EMV ISO-14443 / ISO-7816... }
hf        { High frequency commands... }
hw        { Hardware commands... }
lf        { Low frequency commands... }
nfc       { NFC commands... }
piv       { PIV commands... }
reveng    { CRC calculations from RevEng software... }
smart     { Smart card ISO-7816 commands... }
script    { Scripting commands... }
trace     { Trace manipulation... }
wiegand   { Wiegand format manipulation... }
----- General -----
clear     Clear screen
hints     Turn hints on / off
msleep    Add a pause in milliseconds
rem       Add a text line in log file
quit      Exit program
exit

[offline] pm3 -> [offline] pm3 -> trace list -t mf
```

```

kali@kali: /mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel

help      Use '<command> help' for details of a command
prefs     { Edit client/device preferences ... }
          Technology
analyse   { Analyse utils... }
data      { Plot window / data buffer manipulation ... }
dmv       { BMW 150-1445 / 150-7816 ... }
hf        { High frequency commands ... }
hw        { Hardware commands ... }
lf        { Low frequency commands ... }
nfc       { NFC commands ... }
piv       { PIV commands ... }
reveng    { CRC calculations from RevEng software ... }
smart     { Smart card 150-7816 commands ... }
script    { Scripting commands ... }
trace     { Trace manipulation ... }
wiegand   { Wiegand format manipulation ... }
          General

clear     Clear screen
hints    Turn hints on / off
nsleep   Add a pause in milliseconds
rem      Add a text line in log file
quit     Exit program
exit     Exit program

[offline] pm3 -> trace load -f /mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel/grand-classic.trace
[-] Loaded 7849 bytes from binary file "/mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel/grand-classic.trace"
[-] Recorded Activity (Tracelen = 7849 bytes)
[?] try 'trace list -l -t ...' to view trace. Remember the '-l' param
[offline] pm3 -> trace list -l -t -f
[-] Recorded activity ( 7849 bytes )
[-] start = start of start frame, end = end of frame, src = source of transfer.
[-] 15014443A - all times are in carrier periods (1/13.56MHz)

      Start      End | Src | Data (i denotes parity error) | CRC | Annotation
-----
0          902 | Rdr | 152(7) | | WUPA
2660224    2661216 | Rdr | 152(7) | | WUPA
5320800    5321072 | Rdr | 152(7) | | WUPA
5322224    5324692 | Tag | 04 00 | | WUPA
5332112    5336880 | Rdr | 150 00 57 CD | ok | HALT
6204064    6205056 | Rdr | 152(7) | | WUPA
6206300    6208676 | Tag | 04 00 | | WUPA
6216080    6218544 | Rdr | 193 20 | | ANTICOLL
6219732    6225620 | Tag | 46 43 53 43 15 | | 
6219732    6219732 | Rdr | 152(7) | | WUPA
6219732    6219732 | Rdr | 152(7) | | WUPA
8115572    8117940 | Tag | 04 00 | | WUPA
8125344    8130112 | Rdr | 150 00 57 CD | ok | HALT
8997280    8998272 | Rdr | 152(7) | | WUPA
8999524    9001892 | Tag | 04 00 | | WUPA
9009296    9011760 | Rdr | 193 20 | | ANTICOLL
9012948    9018836 | Tag | 46 43 53 43 15 | | 
9026176    9035784 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
9037892    9041412 | Tag | 08 B6 D0 | ok | 
9108256    9113024 | Rdr | 150 00 57 CD | ok | HALT

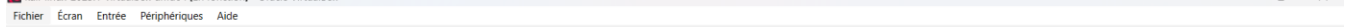
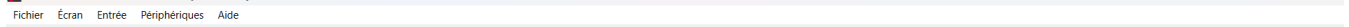
```

```

kali@kali: /mnt/Share/FCSC2025/Hardware/Grand_Classic_Hotel

6204064    6205056 | Rdr | 152(7) | | WUPA
6206300    6208676 | Tag | 04 00 | | WUPA
6216080    6218544 | Rdr | 193 20 | | ANTICOLL
6219732    6225620 | Tag | 46 43 53 43 15 | | 
6219732    6219732 | Rdr | 152(7) | | WUPA
6219732    6219732 | Rdr | 152(7) | | WUPA
8115572    8117940 | Tag | 04 00 | | WUPA
8125344    8130112 | Rdr | 150 00 57 CD | ok | HALT
8997280    8998272 | Rdr | 152(7) | | WUPA
8999524    9001892 | Tag | 04 00 | | WUPA
9009296    9011760 | Rdr | 193 20 | | ANTICOLL
9012948    9018836 | Tag | 46 43 53 43 15 | | 
9026176    9035784 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
9037892    9041412 | Tag | 08 B6 D0 | ok | 
9108256    9113024 | Rdr | 150 00 57 CD | ok | HALT
9157680    9158800 | Rdr | 152(7) | | WUPA
9160036    9162404 | Tag | 04 00 | | WUPA
9169824    9180352 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
9181540    9185060 | Tag | 08 B6 D0 | ok | 
9224528    9229296 | Rdr | 150 00 57 CD | ok | HALT
9278704    9279696 | Rdr | 152(7) | | WUPA
9280932    9283300 | Tag | 04 00 | | WUPA
9290720    9301248 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
9302420    9305940 | Tag | 08 B6 D0 | ok | 
9353872    9357840 | Rdr | 160 04 01 30 | ok | AUTH:A(+)
9359796    9364532 | Tag | 06 88 47 A7 | | AUTH: nt (lfsr16 index 10004)
9375456    9384832 | Rdr | e01 20 C6 75 1A AA 7F A2 | | AUTH: nr ar (enc)
9551968    9556672 | Rdr | c01 28 72 5d1 | | DEC(+)
9733872    9738576 | Rdr | 90 291 c01 a21 | | 
9915200    9919904 | Rdr | 1571 8a1 ec1 251 | | 
10085680    10090384 | Rdr | 1791 CE 7 14 | | 
10270960    10275664 | Rdr | 183 56 781 74 | | 
10450144    10455912 | Rdr | 150 00 57 CD | ok | HALT
10580352    10581344 | Rdr | 152(7) | | WUPA
10582596    10584964 | Tag | 04 00 | | WUPA
10592244    10600212 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
10604084    10607604 | Tag | 08 B6 D0 | ok | 
10659328    10664096 | Rdr | 150 00 57 CD | ok | HALT
10714400    10715472 | Rdr | 152(7) | | WUPA
10716708    10719076 | Tag | 04 00 | | WUPA
10726496    10737024 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
10738212    10741732 | Tag | 08 B6 D0 | ok | 
10824592    10831360 | Rdr | 08 05 01 3D | ok | AUTH:A(+)
10835316    10840052 | Tag | D6 8D 77 23 | | AUTH: nt (lfsr16 index 20254)
10850912    10860288 | Rdr | c41 68 F8 4E 131 29 cc1 9a1 | | AUTH: nr ar (enc)
11020800    11032848 | Rdr | 159 891 c41 89 | | 
11210272    11214976 | Rdr | 13c1 68 b11 7E | | READ SIG
11384160    11388864 | Rdr | 1691 45 CF 371 | | AUTH-69(69)
11551632    11556400 | Rdr | 1571 AC c11 001 | | 
11725648    11730352 | Rdr | 1141 d31 DC C0 | | 
11896096    11900864 | Rdr | 150 00 57 CD | ok | HALT
11941232    11942224 | Rdr | 152(7) | | WUPA
11942476    11945044 | Tag | 04 00 | | WUPA
11953248    11963776 | Rdr | 193 70 46 43 53 43 15 F9 57 | ok | SELECT_UID
11964964    11968484 | Tag | 08 B6 D0 | ok | 

```



4.1 Données extraites

- Bloc 4 (16 octets de données, CRC ignoré) : 46 43 53 43 7B 64 63 61 34 31 62 61 66 65 34 38
- Bloc 5 (16 octets de données, CRC ignoré) : 63 35 37 62 66 32 63 39 33 30 39 63 34 38 35 64
- Bloc 6 (16 octets de données, CRC ignoré) : 61 32 36 37 64 32 33 64 65 30 34 66 39 7D 00 00

Interprétation ASCII : bloc 4 = "FCSC{dca41baf48" ; bloc 5 = "c57bf2c9309c485d" ; bloc 6 = "a267d23de04f9}\x00\x00".

4.2 Flag

En concaténant les 3 payloads de 16 octets (blocs 4, 5 et 6), en ignorant les 2 octets CRC de chaque réponse, puis en supprimant le padding 0x00 final, on obtient : FCSC{dca41baf48c57bf2c9309c485da267d23de04f9}.

5. Récupération de la clé (si non triviale)

Si « trace decrypt -k ... » ne fonctionne pas, la clé de secteur doit être récupérée. Dans MIFARE Classic, l'authentification Crypto1 (AUTH) produit des nonces (Nt/Nr/Ar/At) exploitables : une réutilisation de nonce ou des paramètres faibles permettent l'attaque mfkey32 (classique en CTF).

- Extraire les tuples (UID, Nt, Nr, Ar, At) pour un bloc du secteur ciblé.
- Tester mfkey32 (et variantes) pour reconstruire Key A / Key B.
- Réinjecter la clé dans « trace decrypt -k <KEY> », puis relister « trace list -1 -t mf ».

6. Tableau récapitulatif des preuves

Le tableau suivant synthétise les éléments observables dans la trace (ISO 14443-A, AUTH, READBLOCK) et les champs utiles. Il est conservé tel quel depuis le document source.

Preuve / Champ	Valeur retenue	Justification
ATQA	04 00	ATQA après WUPA/REQA : carte Type A.
UID (CL1)	46 43 53 43	UID issu de l'anticollision/select (93 20 / 93 70 ...).
SAK	0x08	SAK=0x08 : cohérent avec MIFARE Classic 1K.

AUTH-A(4)	60 04 ...	Authentification Key A sur le bloc 4 (début de session CRYPTO1).
Clé utilisée	FFFFFFFFFFFF	Clé MIFARE par défaut observée ; utilisée pour trace decrypt.
Bloc 4 (data)	46 43 53 43 7B 64 63 61 34 31 62 61 66 65 34 38	ASCII : "FCSC{dca41baf48".
Bloc 5 (data)	63 35 37 62 66 32 63 39 33 30 39 63 34 38 35 64	ASCII : "c57bf2c9309c485d".
Bloc 6 (data)	61 32 36 37 64 32 33 64 65 30 34 66 39 7D 00 00	ASCII : "a267d23de04f9}" + padding.

7. Commandes pm3 ciblées (blocs 4/5/6/7)

But : faire apparaître (et filtrer) les opérations MIFARE Classic READBLOCK(4/5/6) contenant le flag, à partir d'une trace Proxmark3 (sans matériel).

Important :

- Ne recopiez pas le préfixe du prompt (« [offline] pm3 --> ») : tapez uniquement la commande.
- Selon les versions, `trace list` n'accepte pas l'option `-v`. Utilisez `trace list --help` si besoin.
- Pour "voir" uniquement les blocs 4/5/6/7, le plus simple est d'exporter la sortie puis de la filtrer hors de pm3 (grep).

```
trace load -f grand-classic.trace
trace list -1 -t 14a
trace list -1 -t mf
trace decrypt -k FFFFFFFFFFFFFF # si la clé est connue (défaut CTF très fréquent)
trace list -1 -t mf # relire après déchiffrement
# Option : exporter puis filtrer hors pm3
# pm3 -c "trace load -f grand-classic.trace" -c "trace decrypt -k FFFFFFFFFFFFFF" -c "trace list -1 -t mf"
> pm3_output.txt
# grep -n "READBLOCK(4)\|READBLOCK(5)\|READBLOCK(6)\|READBLOCK(7)" -n pm3_output.txt
```

Lecture des réponses READBLOCK(i) et extraction du "payload" (sans CRC) :

- Sur MIFARE Classic, une réponse à READ (0x30) contient 16 octets de données suivis de 2 octets CRC_A (soit 18 octets au total).
- Dans `trace list -1 -t mf`, chaque READBLOCK(i) est généralement suivi d'une ligne "Tag" (et parfois d'une ligne marquée `\*`) qui affiche ces octets.
- Pour reconstruire le contenu du bloc, conservez uniquement les 16 premiers octets et ignorez les 2 derniers (CRC).

Pourquoi le flag est précisément dans les blocs 4-6 :

- La carte est une MIFARE Classic 1K (SAK=0x08). Sa mémoire est organisée en 16 secteurs de 4 blocs.
- Le secteur 0 (blocs 0-3) contient notamment le “manufacturer block” (bloc 0) et est rarement utilisé pour un secret applicatif en CTF.
- Le premier secteur applicatif est le secteur 1 (blocs 4-7) : blocs 4-6 = données, bloc 7 = “sector trailer” (clés + access bits).
- La trace montre une authentification sur le bloc 4 (AUTH-A(4)), ce qui ouvre une session Crypto1 pour le secteur 1 ; il est alors logique que le lecteur lise ensuite les blocs 4, 5 et 6.
- Le flag ASCII fait ~46 octets (préfixe “FCSC{” + 40 hex + “}”) : il tient naturellement sur 3 blocs de 16 octets, donc blocs 4-6.

Annexe A – Captures d’écran et sorties brutes

Cette annexe conserve l’ensemble des captures présentes dans le document d’origine, sans suppression. Elles servent de preuve et de support pédagogique. Les textes ont été condensés dans le corps du rapport.

Dans la sortie de ``trace list -1 -t mf``, chercher ``READBLOCK(4)``, ``READBLOCK(5)``, ``READBLOCK(6)``. La ligne marquée ``*`` immédiatement associée à chaque `READBLOCK` correspond à la réponse en clair : 16 octets de données + 2 octets de CRC_A. On conserve uniquement les 16 premiers octets (payload).

- Reconstitution du flag.

Concaténer les payloads des blocs 4, 5 et 6 ($3 \times 16 = 48$ octets), supprimer le padding ``0x00`` en fin de chaîne, puis décoder en ASCII. On obtient une chaîne de type ``FCSC{...}``.

Pourquoi le flag est dans les blocs 4 à 6

Sur une carte MIFARE Classic 1K, la mémoire est organisée en secteurs de 4 blocs de 16 octets. Le secteur 0 (blocs 0–3) contient l'UID et des données fabricant ; son bloc 3 est un "sector trailer" (clés + droits). Le secteur 1 correspond aux blocs 4–7 : le bloc 7 est aussi un trailer, et les blocs 4–6 sont donc des blocs de données. Dans ce challenge, la trace montre une authentification ciblant le bloc 4 (donc le secteur 1), suivie de lectures READ sur les blocs 4, 5 et 6 ; c'est précisément l'endroit naturel où placer des données applicatives, et la taille du flag tient sur 3 blocs (≈ 46 octets pour ``FCSC{`` + 40 hex + ``}``), d'où la reconstruction à partir de 4–6.

Si la clé n'est pas connue

Deux chemins existent : (A) la trace révèle l'usage d'une clé par défaut (souvent ``FFFFFFFFFFFF``) et le déchiffrement suffit ; (B) sinon, extraire un tuple d'authentification (UID, NT, NR, AR, AT) depuis la trace et dériver la clé via ``mfkey32`/`mfkey32v2``, puis relancer ``trace decrypt -k <CLE>`` et relister ``-t mf``.